



UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI
INGEGNERIA ELETTRICA ELETTRONICA ED INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

RELAZIONE TECNICA

HW2 - MICROSERVIZI PER GESTIONE UTENTI E MONITORAGGIO VOLI
AEREI

Alunni:

*Arena Giorgia
Tornabene Alessio*

Docenti:

*Di Stefano Antonella
Morana Giovanni*

PhD:

Genovese Alessandro

INDICE

1	INTRODUZIONE E OBIETTIVI.....	3
	1.1 PANORAMICA DEL SISTEMA	3
	1.2 OBIETTIVI PROGETTUALI	3
2	ARCHITETTURA DEL SISTEMA.....	6
	2.1 API GATEWAY & ENTRY POINT	6
	2.2 MICROSERVIZI.....	6
	2.3 INFRASTRUTTURA DATI E COMUNICAZIONE	7
3	DETTAGLIO DEI COMPONENTI E SCELTE IMPLEMENTATIVE.....	10
	3.1 USER MANAGER & GESTIONE DELLA CONCORRENZA.....	10
	3.2 DATA COLLECTOR & CIRCUIT BREAKER	10
	3.3 ALERT SYSTEM & PROCESSING DETERMINISTICO	12
	3.4 NOTIFIER & GESTIONE AT-LEAST-ONCE.....	12
4	ANALISI DELLE TECNOLOGIE E PROTOCOLLI.....	14
	4.1 ETEROGENEITÀ DEI PROTOCOLLI E STRATEGIE DI COMUNICAZIONE	14
	4.2 PERSISTENZA POLIGLOTTA (POLYGLOT PERSISTENCE)	15
	4.3 CORE INFRASTRUTTURALE: APACHE KAFKA E GARANZIE DI CONSEGNA	15
5	ANALISI DEL COMPORTAMENTO IN SCENARI CRITICI (FAILURE ANALYSIS)	20
	5.1 SCENARIO A: INDISPONIBILITÀ DELL'API OPENSKY.....	20
	5.2 SCENARIO B: CRASH E RIAVVIO DEL NOTIFIER	20
	5.3 SCENARIO C: RACE CONDITION ALL'AVVIO (COLD START)	21
6	ISTRUZIONI PER L'ESECUZIONE E TESTING API.....	22
	6.1 AVVIO DELL'INFRASTRUTTURA	22
	6.2 GUIDA AGLI ENDPOINT (API REFERENCE)	22
	6.3 VERIFICA DEL FLUSSO DI ALERTA (END-TO-END TEST)	24
7	CONCLUSIONI.....	29

1 INTRODUZIONE E OBIETTIVI

1.1 Panoramica del Sistema

Il seguente progetto consiste nello sviluppo di un'architettura a microservizi distribuita per il monitoraggio in tempo reale del traffico aereo, integrata con le API pubbliche di *OpenSky Network*. Il sistema è progettato per consentire agli utenti di registrare interessi specifici (monitoraggio di un determinato aeroporto) e definire soglie personalizzate di traffico aereo (*high_value* e *low_value*). Al superamento di tali soglie, il sistema innesca una pipeline di elaborazione eventi che culmina nell'invio di notifiche via e-mail agli utenti iscritti.

L'intera infrastruttura è orchestrata tramite **Docker Compose**, garantendo isolamento dei processi e portabilità. L'accesso ai servizi è mediato da un **API Gateway (NGINX)**, che funge da *Reverse Proxy* esponendo un unico punto di ingresso (Porta 80) e mascherando la complessità della rete interna.

1.2 Obiettivi Progettuali

L'obiettivo primario non si limita alla semplice funzionalità, ma pone uno specifico focus su:

- 1. Disaccoppiamento e Comunicazione Asincrona** Per garantire che il fallimento di un componente non comprometta l'intero sistema (*Loose Coupling*), è stato introdotto **Apache Kafka** come Message Broker centrale. La comunicazione tra il *Data Collector*, l'*Alert System* e il *Notifier* avviene in modalità asincrona tramite pattern *Publish/Subscribe* su topic dedicati (*to-alert-system*, *to-notifier*). Questo permette di gestire picchi di carico e garantisce che i messaggi vengano processati anche se il consumatore è temporaneamente indisponibile.
- 2. Resilienza e Tolleranza ai Guasti (Fault Tolerance)** Il sistema implementa pattern avanzati per la gestione degli errori, evidenziati nell'analisi del codice:

- o **Circuit Breaker Pattern:** Nel microservizio *Data Collector*, le chiamate verso l'API esterna di OpenSky sono protette da un *Circuit Breaker* personalizzato. Questo meccanismo previene il sovraccarico del sistema e gestisce i fallimenti transitori dell'API esterna aprendo il circuito dopo una soglia di errori definita (*failure_threshold=4*) e tentandone il ripristino dopo un timeout (*recovery_timeout=60*).
 - o **Active Waiting:** I servizi *Alert System* e *Notifier* implementano logiche di *active waiting* (*wait_for_kafka*), impedendo il crash dell'applicazione all'avvio qualora il broker Kafka non sia ancora pronto a ricevere connessioni.
 - o **Retry Policies:** Sono state configurate politiche di ri-tentativo automatico sia a livello di connessione database (Mongo e Postgres) sia a livello di *Kafka Producer* (*retries=3*), per mitigare instabilità di rete temporanee.
3. **Coerenza ed Integrità dei Dati** Per assicurare che nessun allarme critico vada perso, la configurazione dei Producer Kafka privilegia la **durabilità** rispetto alla **latenza** pura, utilizzando l'impostazione **acks='all'**. Inoltre, per gestire la semantica *At-Most-Once* e prevenire l'invio di notifiche duplicate in caso di riavvii dei container, sono stati implementati meccanismi di deduplica basati su ID univoci (hash MD5 generati da timestamp e dati di volo) sia nell'*Alert System* che nel *Notifier*.
4. **Efficienza ed Eterogeneità dei Protocolli** L'architettura sfrutta protocolli diversi in base alla natura dell'interazione:
- o **REST (HTTP):** Utilizzato per l'interazione sincrona con l'utente finale (tramite Flask) per la massima compatibilità.
 - o **gRPC (Protocol Buffers):** Utilizzato per la comunicazione interna ad alte prestazioni tra *Data Collector* e *User Manager* (funzioni CheckUser, DeleteData), garantendo bassa latenza e tipizzazione forte dei dati scambiati.

In sintesi, il progetto dimostra come l'orchestrazione di tecnologie eterogenee (MongoDB, PostgreSQL, Kafka,...) possa convergere in un sistema robusto, scalabile e resistente ai guasti.

2 ARCHITETTURA DEL SISTEMA

L'architettura del sistema è progettata secondo il paradigma a **Microservizi**, orchestrata tramite **Docker Compose**, che definisce e gestisce il ciclo di vita di 8 container distinti. La comunicazione tra i servizi avviene tramite un mix di protocolli sincroni (HTTP REST, gRPC) e asincroni (Kafka), scelti in base ai requisiti di latenza e disaccoppiamento di ogni componente.

Il sistema si articola nei seguenti moduli funzionali:

2.1 API Gateway & Entry Point

- **NGINX Gateway:** Agisce come unico punto di ingresso per il traffico esterno sulla porta **80**. Configurato come *Reverse Proxy*, instrada le richieste HTTP ai microservizi competenti in base all'URL, garantendo l'incapsulamento della logica interna:
 - `/users` **User Manager Service** (Porta 5000).
 - `/interests` e `/flights` **Data Collector Service** (Porta 5001).

2.2 Microservizi

1. **User Manager Service:**

- *Ruolo:* Gestisce la registrazione, l'autenticazione e la cancellazione degli utenti.
- *Tecnologia:* Python (Flask) + gRPC.
- *Interfacce:* Espone API REST sulla porta 5000 per il client e un server **gRPC** sulla porta 50051 per controlli interni ad alta performance.
- *Persistenza:* Utilizza **PostgreSQL** per memorizzare in modo relazionale le credenziali utente.
- *Ottimizzazione:* Implementa una **Cache in-memory** per garantire l'idempotenza delle richieste (semantica *At-Most-Once*).

2. **Data Collector Service:**

- o *Ruolo*: Gestisce gli interessi degli utenti (aeroporti da monitorare) e l'acquisizione dati. Interroga periodicamente l'API esterna *OpenSky Network*.
- o *Tecnologia*: Python (Flask).
- o *Interfacce*: Espone API REST sulla porta 5001 e agisce sia come client gRPC (verso User Manager) che, come server gRPC, (porta 50052) per la gestione della cancellazione dati a cascata.
- o *Resilienza*: Implementa il pattern **Circuit Breaker** per proteggere il sistema da malfunzionamenti dell'API esterna.
- o *Output*: Produce messaggi sul topic Kafka *to-alert-system*.

3. **Alert System:**

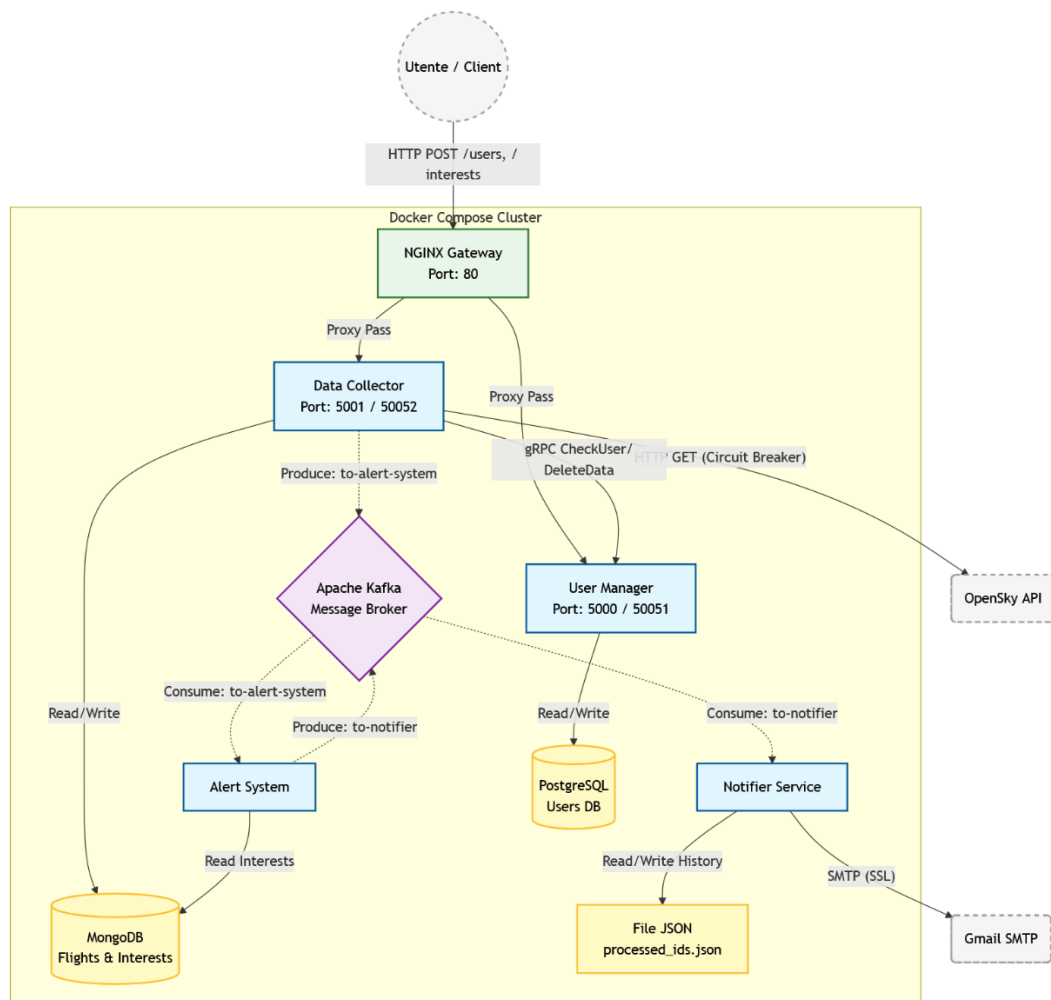
- o *Ruolo*: Motore di elaborazione eventi (Event Processor). Consuma i dati di volo, recupera le soglie utente dal database e valuta le condizioni di allarme.
- o *Tecnologia*: Python (Consumer/Producer Kafka).
- o *Logica*: Implementa una deduplica temporale per evitare l'invio continuo di notifiche.
- o *Output*: In caso di violazione soglie, produce eventi di allarme sul topic *to-notifier*.

4. **Notifier Service:**

- o *Ruolo*: Consuma gli eventi di allarme e invia e-mail formattate agli utenti tramite server SMTP (Gmail).
- o *Tecnologia*: Python.
- o *Affidabilità*: Utilizza una memoria locale per tracciare gli ID processati e prevenire invii duplicati in caso di rilettura dei messaggi Kafka.

2.3 Infrastruttura Dati e Comunicazione

- **Apache Kafka:** Backbone di comunicazione asincrona. Disaccoppia i produttori dai consumatori, garantendo bufferizzazione e scalabilità. I servizi attendono attivamente la disponibilità del broker all'avvio (*wait_for_kafka*).
- **MongoDB:** Database NoSQL utilizzato dal *Data Collector* e dall'*Alert System* per memorizzare dati non strutturati ad alta frequenza (storico voli) e le configurazioni degli interessi utente.
- **PostgreSQL:** utilizzato dallo *User Manager* per garantire la consistenza transazionale dei dati anagrafici.



2.4 Sicurezza della Rete (Network Isolation):

L'orchestrazione Docker Compose definisce una rete interna privata (*backend-network*). I database (PostgreSQL e MongoDB) e il broker Kafka

non espongono porte all'host esterno (nessun mapping "ports" pubblico, salvo per debug), ma sono raggiungibili esclusivamente dai microservizi applicativi all'interno della rete Docker. L'unica porta esposta pubblicamente è la 80 dell'API Gateway NGINX, riducendo drasticamente la superficie di attacco.

3 DETTAGLIO DEI COMPONENTI E SCELTE IMPLEMENTATIVE

In questa sezione vengono analizzate le soluzioni ingegneristiche adottate per risolvere le criticità tipiche dei sistemi distribuiti, quali la gestione della concorrenza, la tolleranza ai guasti e la consistenza dei dati.

3.1 User Manager & Gestione della Concorrenza

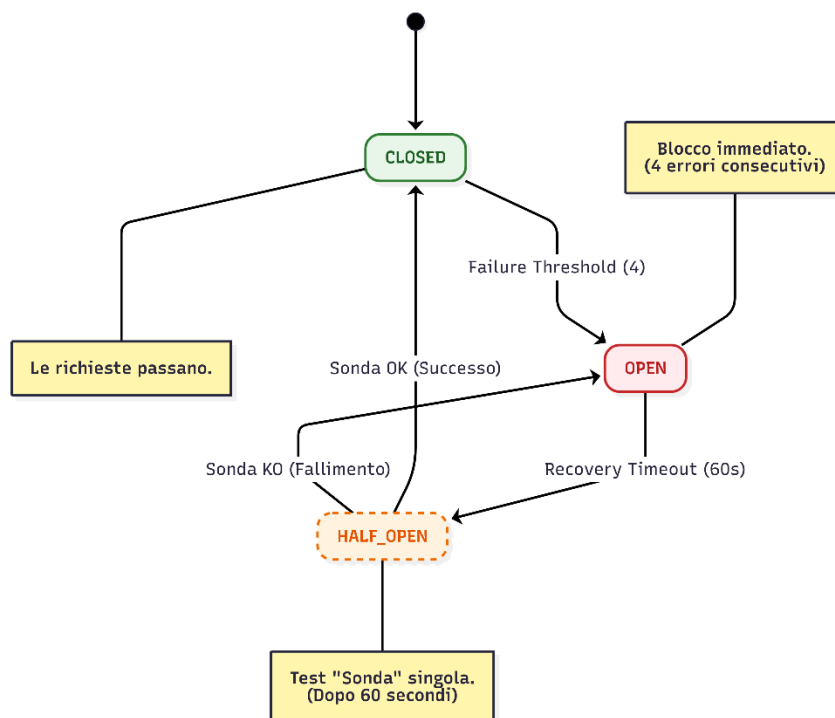
Il microservizio *User Manager* è il punto di riferimento per l'identità e l'autenticazione. Le scelte implementative mirano a proteggere il database relazionale (PostgreSQL) da accessi eccessivi e a garantire la correttezza delle transazioni.

- **Cache Thread-Safe con TTL:** Per ottimizzare i tempi di risposta, è stata sviluppata una classe dedicata *Cache*. A differenza di un dizionario standard non sicuro in ambienti multi-thread, questa implementazione incapsula un *threading.Lock()* per serializzare l'accesso in scrittura e lettura. Inoltre, un thread "daemon" dedicato (*pulisci_cache*) monitora costantemente i timestamp dei dati salvati, rimuovendo automaticamente le entry che superano il *Time-To-Live* configurato (*ttl_seconds=300*), prevenendo memory leak.
- **Idempotenza (Pattern At-Most-Once):** Per gestire eventuali ritrasmissioni involontarie da parte dei client (es. timeout di rete su una POST /users), il servizio analizza l'header HTTP custom Request-ID. Prima di processare una registrazione, il sistema interroga la cache: se l'ID è presente, restituisce immediatamente la risposta memorizzata (codice 200/201 e body JSON) senza rieseguire la query INSERT sul database, garantendo che un utente non venga creato due volte per lo stesso intento.

3.2 Data Collector & Circuit Breaker

Il *Data Collector* opera in un contesto critico, dovendo interagire con un servizio esterno instabile (*OpenSky Network*) e comunicare internamente con protocolli eterogenei.

- **Circuit Breaker Pattern:** Per prevenire il blocco dei thread in attesa di un'API esterna non responsiva, è stato implementato un *Circuit Breaker* personalizzato (*class CircuitBreaker*). La macchina a stati finiti gestisce tre condizioni:
 - o **CLOSED:** Le richieste passano normalmente.
 - o **OPEN:** Dopo *failure_threshold*=4 eccezioni consecutive, il circuito si apre e le richieste vengono respinte immediatamente sollevando una *CircuitBreakerOpenException*.
 - o **HALF_OPEN:** Trascorso il *recovery_timeout*=60 secondi, il sistema lascia passare una singola richiesta "sonda". Se ha successo, il circuito si chiude; altrimenti, torna aperto.



- **Robustezza del Client gRPC:** La comunicazione verso lo *User Manager* utilizza gRPC con una configurazione JSON avanzata per la *Retry Policy*. È stato definito un backoff esponenziale (*initialBackoff*: 0.5s, *backoffMultiplier*: 2) con un massimo di 5 tentativi (*maxAttempts*: 5). Questo assicura che il Data

Collector non fallisca immediatamente se lo User Manager è in fase di riavvio o momentaneamente irraggiungibile.

- **Event Tracing:** Ogni payload inviato a Kafka è arricchito con un campo *source* (valorizzato dinamicamente, es. "Richiesta POSTMAN" o "Aggiornamento voli"). Questo permette, in fase di debugging, di distinguere se un allarme è stato generato da un test manuale o dal ciclo di monitoraggio automatico.

3.3 Alert System & Processing Deterministico

La logica dell' *Alert System* è stata progettata per essere resiliente ai riavvii e consistente nei dati.

- **Strategia "Active Waiting" su Kafka:** Il servizio implementa una logica di sincronizzazione all'avvio per gestire la dipendenza temporale dal broker. Per i dettagli implementativi sulla gestione della race condition iniziale e sul meccanismo di polling, si rimanda all'analisi dello **Scenario C nella Sezione 5.3**.
- **Deduplica Deterministica:** Per evitare notifiche duplicate causate da rielaborazioni di messaggi Kafka (es. mancato commit dell'offset), viene generato un *alert_id* univoco. Questo è calcolato tramite hash MD5 della concatenazione di *email*, *airport*, *timestamp* e *value*. In questo modo, lo stesso evento di volo produrrà sempre lo stesso ID, permettendo al servizio a valle di riconoscerlo come duplicato.

3.4 Notifier & Gestione At-Least-Once

Il Notifier è il consumatore finale della pipeline ed è responsabile dell'invio delle e-mail tramite server SMTP. Per garantire che nessuna notifica vada persa, il servizio opera in regime **At-Least-Once**. Poiché questa modalità potrebbe comportare la riletture di messaggi già processati (in caso di crash), è stato implementato un meccanismo di **Deduplica in-memory**: una *sliding window*

(basata su *deque*) traccia gli ID degli ultimi messaggi gestiti, scartando eventuali duplicati prima dell'invio.

*Nota: Per l'analisi tecnica approfondita sulla strategia di commit manuale e sulla generazione degli ID di deduplica, si rimanda alla **Sezione 4.3**.*

4 ANALISI DELLE TECNOLOGIE E PROTOCOLLI

L'architettura del sistema seleziona protocolli di comunicazione e tecnologie di storage specifici per i requisiti non funzionali (scalabilità, coerenza, latenza) di ciascun microservizio.

4.1 Eterogeneità dei Protocolli e Strategie di Comunicazione

Il sistema orchestra tre diversi paradigmi di comunicazione per bilanciare accessibilità pubblica, performance interna e disaccoppiamento.

1. REST su HTTP (Comunicazione Sincrona Pubblica)

- **Ruolo:** È il protocollo di frontiera gestito dall'API Gateway **NGINX**, configurato come *Reverse Proxy* sulla porta 80.
- **Implementazione:** I servizi *User Manager* e *Data Collector* espongono endpoint RESTful (es. POST /users, POST /interests) implementati tramite **Flask**.
- **Motivazione:** La scelta di REST garantisce l'interoperabilità universale con qualsiasi client esterno e facilita l'integrazione.

2. gRPC con Protocol Buffers (Comunicazione Sincrona Interna)

- **Ruolo:** Gestisce la comunicazione *Inter-Process Communication* (IPC) critica tra *Data Collector* e *User Manager*, specificamente per le procedure CheckUser e DeleteData.
- **Implementazione:** Il contratto di interfaccia è definito in user.proto, che stabilisce messaggi tipizzati (CheckUserRequest). Il client gRPC nel *Data Collector* è stato configurato con una **Service Config** avanzata che definisce una politica di *Retry* con backoff esponenziale (initialBackoff: 0.5s, maxAttempts: 5) per gestire la transitorietà degli errori di rete.

- **Motivazione:** Rispetto a REST, gRPC offre performance superiori grazie alla serializzazione binaria (Protobuf) e alla gestione efficiente delle connessioni HTTP/2.

3. Protocollo Kafka (Streaming Asincrono)

- a. **Ruolo:** Costituisce la spina dorsale per il trasferimento dati ad alto throughput, disaccoppiando i servizi produttori (*Data Collector*, *Alert System*) dai consumatori (*Notifier*).

4.2 Persistenza Poliglotta (Polyglot Persistence)

Il layer di persistenza è partizionato in base alla struttura e alla modalità di accesso ai dati:

1. **PostgreSQL (Relazionale):** Dedicato allo *User Manager* per garantire l'integrità referenziale delle identità utente (users) e prevenire duplicati critici.
2. **MongoDB (NoSQL):** Utilizzato da *Data Collector* e *Alert System* per memorizzare dati eterogenei come i payload dei voli (flights) e le configurazioni degli interessi. Sfrutta la flessibilità dello schema JSON-like per adattarsi ai dati provenienti dalle API esterne.

4.3 Core Infrastrutturale: Apache Kafka e Garanzie di Consegna

La configurazione di Apache Kafka non si limita al trasporto messaggi, ma definisce la strategia di **Fault Tolerance** dell'intero sistema. L'implementazione segue rigorosamente il pattern "**At-Least-Once Stretto**" (High Reliability), preferendo la consistenza dei dati alla latenza pura.

A. Configurazione Producer

Per i produttori critici, la configurazione sacrifica il throughput massimo in favore della certezza di scrittura:

- **acks='all':** Il producer riceve conferma solo quando il messaggio è stato replicato su tutti i broker (nel seguente progetto, si adotta un singolo broker).

Questo garantisce che un crash del leader Kafka non comporti la perdita di dati già confermati.

- **max.in.flight.requests.per.connection=1:** Questa impostazione forza il producer a inviare un solo batch alla volta, garantendo l'ordinamento rigoroso dei messaggi.

B. Semantica di Consumo

Nei consumatori (Alert System e Notifier), è stata disabilitata la gestione automatica degli offset (`enable.auto.commit: False`) per implementare una logica di Commit Manuale Sincrono.

Il ciclo di elaborazione segue una sequenza atomica "Fetch-Process-Flush-Commit":

1. **Process:** Elaborazione della business logic e generazione dell'evento di output.
2. **Produce & Flush:** Chiamata bloccante a `producer.flush()`. Questo assicura che l'evento derivato (es. la notifica) sia fisicamente salvato su Kafka *prima* di avanzare l'offset.
3. **Sync Commit:** Chiamata a `consumer.commit(asynchronous=False)`. Solo a questo punto il sistema conferma la lettura.

Questa sequenza garantisce che, in caso di crash in qualsiasi punto intermedio, il messaggio originale venga riletto al riavvio, eliminando il rischio di perdita dati.

C. Idempotenza e Deduplica Applicativa

L'adozione della semantica *At-Least-Once* introduce intrinsecamente il rischio di duplicati (messaggi riprocessati dopo un crash prima del commit dell'offset). Per risolvere questo problema architetturale è stata implementata una strategia di **Deduplica Applicativa** su due livelli:

1. **Generazione ID Deterministico (Alert System):** Ogni evento di allarme viene marcato con un *alert_id* univoco, generato tramite hash MD5 della concatenazione di `{email, airport, timestamp, value}`. Questo ID funge da

"impronta digitale" immutabile: lo stesso evento produrrà sempre lo stesso hash, indipendentemente da quante volte viene riprocessato.

2. **Filtro Notifier:** Il consumatore mantiene una memoria a breve termine (deque con `maxlen=10000`) degli hash processati. Prima di contattare il server SMTP, verifica l'esistenza dell'ID in questa lista. Se presente, il messaggio viene scartato silenziosamente (poiché già inviato in precedenza), realizzando di fatto una semantica **Exactly-Once** simulata a livello applicativo.

Per chiarire ulteriormente l'interazione temporale tra i componenti e le garanzie di integrità dei dati discusse sopra, il seguente Sequence Diagram (Figura 4.1) dettaglia il flusso di un singolo pacchetto dati attraverso la pipeline.

Il diagramma evidenzia in particolare la sequenza atomica implementata nell'Alert System (Fase 2): la chiamata bloccante *producer.flush()* viene eseguita rigorosamente prima del *commit()* dell'offset, garantendo che nessun allarme venga perso anche in caso di crash immediato del processo. Infine, viene mostrato come il Notifier (Fase 3) utilizzi la memoria locale per filtrare i duplicati logici, realizzando di fatto una semantica Exactly-Once a livello applicativo.

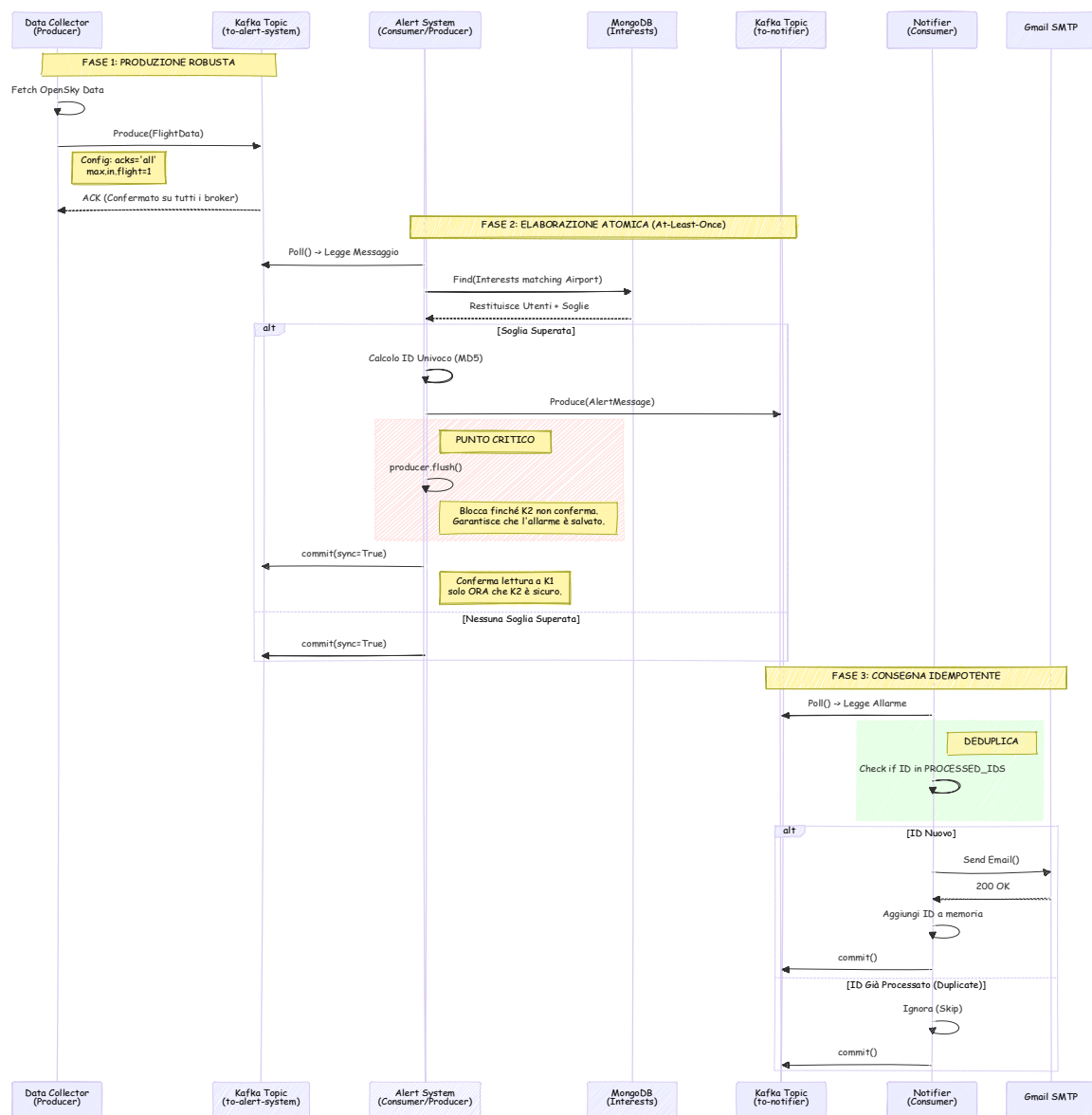


Figura 1-Sequence Diagram (Figura 4.1)

D. Resilienza allo Startup (Active Waiting)

Per garantire che l'applicazione sopravviva ai tempi di latenza nell'elezione del controller Kafka, i consumer implementano pattern di **Active Waiting**. Questa scelta architetturale previene crash fatali durante il bootstrap dell'infrastruttura (si veda l'analisi dettagliata in **Sez. 5.3**).

E. Scalabilità Orizzontale (Consumer Groups)

Una delle motivazioni critiche per l'adozione di Kafka rispetto a una coda messaggi tradizionale risiede nella gestione dei **Consumer Groups**.

Attualmente, il servizio *Alert System* opera come singolo consumatore. Tuttavia, l'architettura è predisposta per lo scaling orizzontale: qualora il carico di dati di volo dovesse aumentare (es. monitoraggio globale), basterà avviare nuove istanze del container *Alert System* appartenenti allo stesso *group.id*. Kafka distribuirà automaticamente le partizioni del topic tra le istanze disponibili, parallelizzando il carico di lavoro senza modificare una riga di codice. Questa caratteristica garantisce che il sistema possa crescere linearmente all'aumentare del throughput.

5 ANALISI DEL COMPORTAMENTO IN SCENARI CRITICI (FAILURE ANALYSIS)

In questa sezione viene analizzato il comportamento dinamico del sistema a fronte di guasti reali simulati, dimostrando l'efficacia delle strategie di robustezza implementate.

5.1 Scenario A: Indisponibilità dell'API OpenSky

Evento: L'API esterna di OpenSky smette di rispondere (timeout) o restituisce errori 5xx per un periodo prolungato.

Reazione del Sistema:

1. **Rilevamento:** Il *Circuit Breaker* nel Data Collector registra i fallimenti consecutivi. Al 4° errore, scatta lo stato **OPEN**.
2. **Protezione:** Le chiamate successive vengono bloccate istantaneamente, salvaguardando i thread del container.
3. Continuità (Fallback): Invece di bloccare la pipeline, il sistema attiva automaticamente la generazione di Dati Mock (simulati).

Risultato: L'utente continua a ricevere notifiche (basate sui dati simulati) e il sistema di monitoraggio non si arresta, garantendo la continuità anche in assenza del fornitore dati.

5.2 Scenario B: Crash e Riavvio del Notifier

Evento: Il container del Notifier crasha improvvisamente mentre sta elaborando una coda di allarmi.

Reazione del Sistema:

1. **Nessuna Perdita Dati:** Grazie alla configurazione `enable.auto.commit: False`, l'offset di Kafka non è stato committato per i messaggi in elaborazione.
2. **Recovery:** Al riavvio, il Notifier rilegge i messaggi non confermati (semantica *At-Least-Once*).

3. **Prevenzione Duplicati:** Se un messaggio era stato inviato ma non committato, il filtro di deduplica (PROCESSED_IDS) riconosce l'ID univoco e impedisce un secondo invio via e-mail.

Risultato: Il sistema garantisce che ogni allarme venga processato, senza generare duplicati per l'utente finale.

5.3 Scenario C: Race Condition all'Avvio

Evento: L'intera infrastruttura viene avviata simultaneamente (docker-compose up). Il container Kafka impiega circa 20-30 secondi per diventare operativo (elezione leader, creazione topic), mentre i microservizi Python completano l'avvio in pochi secondi.

Reazione del Sistema:

1. **Active Waiting:** I servizi *Alert System* e *Notifier* intercettano l'eccezione di connessione ed entrano in un ciclo di polling (wait_for_kafka), interrogando ciclicamente i metadati del cluster tramite list_topics().
2. **Sincronizzazione:** Il ciclo continua (con backoff temporale) finché il broker non risponde positivamente. Solo a quel punto i servizi inizializzano i Consumer.

Risultato: Nessun container termina con codice di errore durante il bootstrap, eliminando la necessità di riavvii manuali.

6 ISTRUZIONI PER L'ESECUZIONE E TESTING API

Il sistema è completamente containerizzato e orchestrato tramite Docker Compose. Di seguito sono riportate le procedure per il deployment e la guida all'utilizzo delle API REST esposte dall'API Gateway.

6.1 Avvio dell'Infrastruttura

Per effettuare la build delle immagini e avviare i container in modalità detached (background):

`docker-compose up --build -d`

Nota sulla fase di Bootstrap:

Immediatamente dopo l'avvio, i servizi Python (Alert System, Notifier) entreranno in uno stato di Active Waiting (log: In attesa di Kafka...). Il sistema sarà pienamente operativo dopo circa 20-30 secondi, tempo necessario al cluster Kafka per l'elezione del controller e l'inizializzazione dei topic.

Per verificare lo stato dei servizi e visualizzare i log in tempo reale:

`docker-compose logs -f [nomedelservizio]`

6.2 Guida agli Endpoint (API Reference)

L'accesso al sistema avviene esclusivamente tramite la porta **80** (NGINX), che agisce da Reverse Proxy instradando le richieste ai microservizi competenti.

Di seguito i payload per testare il sistema (es. tramite Postman o cURL).

A. Registrazione Utente (User Manager)

- **Endpoint:** POST `http://localhost/users`
- **Descrizione:** Crea un nuovo utente su PostgreSQL. Richiede un header custom per garantire l'idempotenza (At-Most-Once).
- **Headers:** Request-ID: `<uuid-univoco>` (Obbligatorio per la cache applicativa).

- **Body:**

JSON

```
{  
  "email": "tornabenealeccio@gmail.it",  
  "password": "datiDistribuiti2025/2026",  
  "nome": "Alessio",  
  "cognome": "Tornabene"  
}
```

B. Aggiunta Interesse di Monitoraggio (Data Collector)

- **Endpoint:** POST `http://localhost/interests`
- **Descrizione:** Registra un aeroporto da monitorare. Questa chiamata innesca internamente una **richiesta gRPC sincrona** verso lo User Manager per validare l'e-mail. Se l'utente esiste, i dati vengono salvati su MongoDB e inizia il monitoraggio.

- **Body:**

- JSON

- {

```
  "email": "tornabenealeccio@gmail.it",  
  "airport": "LIRF",  
  "high_value": 100,  
  "low_value": 20
```

- }

- *Nota: Se $high_value \leq low_value$, l'API restituirà un errore 400 (Bad Request).*

C. Consultazione Dati Volo (Data Collector)

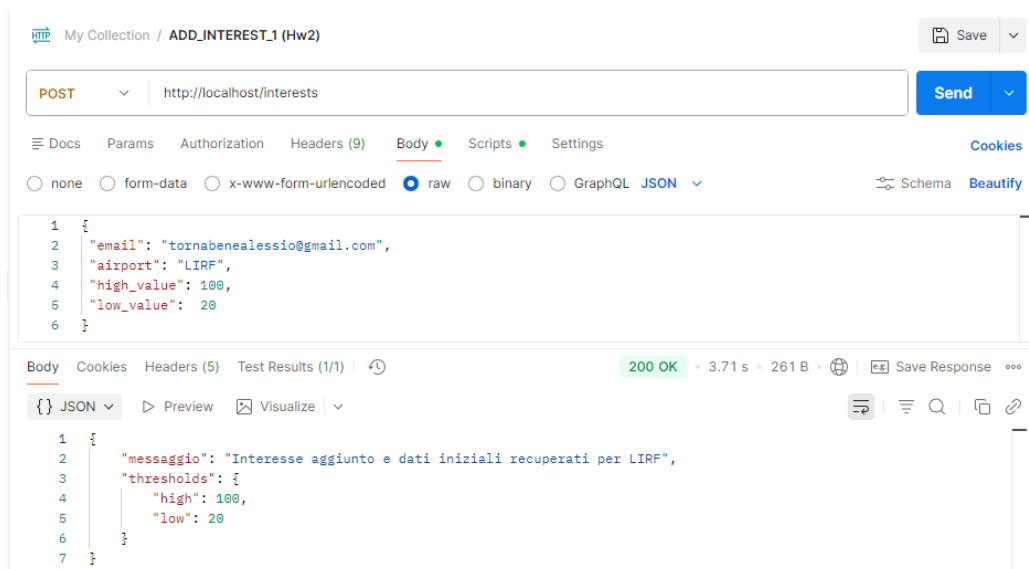
- **Endpoint:** GET `http://localhost/flights/last?airport=LIRF`
- **Descrizione:** Restituisce l'ultimo dato di volo scaricato da OpenSky (o il dato Mock se il circuito è aperto) salvato su MongoDB.

6.3 Verifica del Flusso dell' Alert System

Abbiamo analizzato **QUATTRO** scenari d'uso: i primi tre riguardano l'acquisizione dati (**Real-Time** su richiesta su richiesta POSTMAN), mentre l'ultimo riguarda il monitoraggio ciclico (**Batch**):

CASO 1: SCENARIO IN CUI IL VALORE DEI VOLI E' COMPRESO NEL RANGE VALUE

1. Inviare una POST /interests con una soglia (es. low_value: 20 e high_value: 100).



2. Osservare i log del container **Alert System** con il seguente comando:

docker logs -f alert_system_service

```
Kafka è PRONTO! Connessione stabilita
Alert System OPERATIVO!
Ricevuto da Kafka: LIRF - Voli: 33
█
```

3. Osservare i log del container **Notifier** (Origine: Richiesta POSTMAN) con il seguente comando:

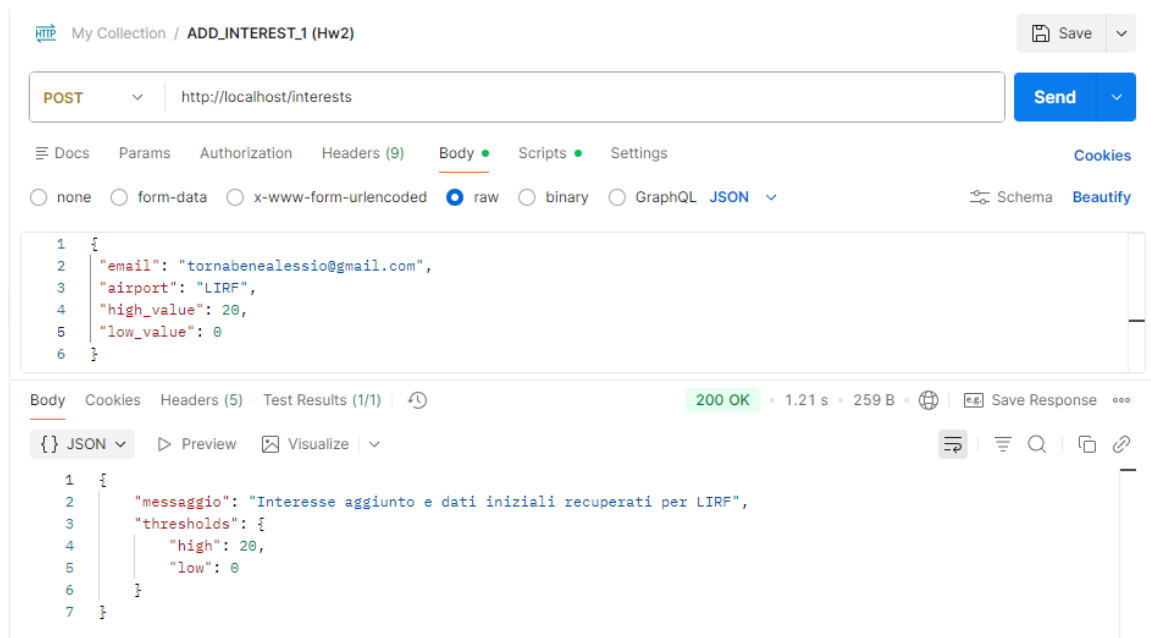
docker logs -f notifier_service

```
Kafka è PRONTO! Connessione stabilita.  
Il consumer Notifier è connesso e sottoscritto al topic.  
█
```

4. Dato che il range dei voli è compreso tra 20 e 100(es. 33) non viene mandata nessuna notifica all'utente tramite e-mail.

CASO 2: SCENARIO IN CUI IL VALORE DEI VOLI SUPERA LA SOGLIA HIGH VALUE

1. Inviare una POST /interests con una soglia (es. high_value:20 e low_value: 0)



2. Osservare i log del container Alert System con il seguente comando:

docker logs -f alert_system_service

```
Ricevuto da Kafka: LIRF - Voli: 34  
Allarme rilevato per tornabenealeccio@gmail.com: Valore ALTO superato  
█
```

3. Osservare i log del container Notifier (Origine: Richiesta POSTMAN) con il seguente comando:

docker logs -f notifier_service

```
Kafka è PRONTO! Connessione stabilita.  
Il consumer Notifier è connesso e sottoscritto al topic.  
Invio con successo a tornabenealeggio@gmail.com  
█
```

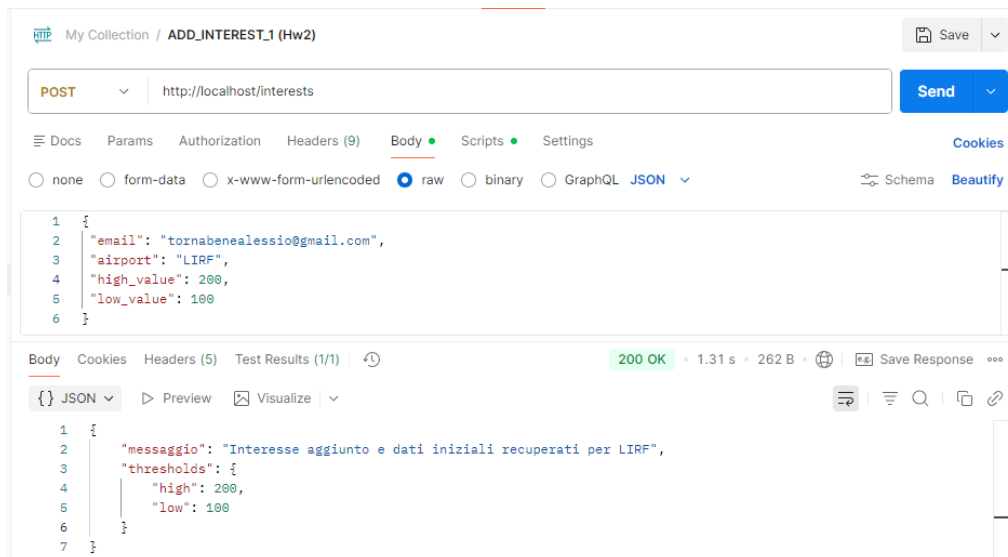
4. Dato che il range dei voli non è compreso tra 0 e 20(es. 34) viene mandata una notifica all'utente tramite e-mail.



NB: notare che l'origine dei dati è POSTMAN per differenziare l'email dal monitoraggio semplice dei voli.

CASO 3: SCENARIO IN CUI IL VALORE DEI VOLI E' AL DI SOTTO DELLA SOGLIA LOW VALUE

1. Inviare una POST /interests con una soglia (es. high_value:200 e low_value: 100)



2. Osservare i log del container Alert System con il seguente comando:

docker logs -f alert_system_service

```
Ricevuto da Kafka: LIRF - Voli: 35
Allarme rilevato per tornabenealelessio@gmail.com: Valore BASSO superato
[]
```

3. Osservare i log del container Notifier (Origine: Richiesta POSTMAN) con il seguente comando:

docker logs -f notifier_service

```
Kafka è PRONTO! Connessione stabilita.
Il consumer Notifier è connesso e sottoscritto al topic.
Invio con successo a tornabenealelessio@gmail.com
```

4. Dato che il range dei voli non è compreso tra 200 e 100(es. 35) viene mandata una notifica all'utente tramite e-mail.

Allerta Voli: LIRF Posta in arrivo x



tornabenealeessio@gmail.com

a me ▼

Ciao tornabenealeessio@gmail.com.

Il sistema di monitoraggio ha rilevato una condizione critica per l'aeroporto: LIRF.

- Origine Dati: Richiesta POSTMAN
- Condizione: Valore BASSO superato
- Valore Attuale Voli: 35
- Soglia Impostata: 100

Ti auguriamo buone vacanze e un grande in bocca al lupo per l'esame di Distributed Systems!

NB: notare che l'origine dei dati è POSTMAN per differenziare l'email dal monitoraggio semplice dei voli.

CASO 4: MONITORAGGIO CICLICO DEI VOLI

Ogni qualvolta vengono aggiornati i voli nel DB(es. ogni 10 min) di un dato aeroporto(es. LIRF) viene mandata una notifica tramite e-mail all'utente.

Allerta Voli: LIRF Posta in arrivo x



tornabenealeessio@gmail.com

a me ▼

Ciao tornabenealeessio@gmail.com.

Il sistema di monitoraggio ha rilevato una condizione critica per l'aeroporto: LIRF.

- Origine Dati: Aggiornamento voli
- Condizione: Valore BASSO superato
- Valore Attuale Voli: 37
- Soglia Impostata: 100

Ti auguriamo buone vacanze e un grande in bocca al lupo per l'esame di Distributed Systems!

NB: notare che l'origine dei dati è AGGIORNAMENTO VOLI per differenziare l'e-mail dal monitoraggio semplice dei voli.

7 CONCLUSIONI

Il progetto ha raggiunto l'obiettivo di realizzare un sistema di monitoraggio distribuito resiliente, scalabile e coerente, superando le criticità tipiche delle architetture a microservizi attraverso un approccio ingegneristico rigoroso.

L'analisi finale del sistema evidenzia tre pilastri fondamentali che ne garantiscono la robustezza operativa ed eterogeneità:

1. Orchestrazione di Protocolli Eterogenei:

La coesistenza di REST, gRPC e Kafka non è stata una scelta stilistica, ma funzionale. Mentre l'uso di gRPC (con Protobuf) ha minimizzato la latenza nella comunicazione sincrona interna, l'adozione di Kafka ha introdotto un disaccoppiamento temporale essenziale. Questo ha permesso di gestire picchi di carico asincroni senza bloccare l'interazione utente, garantendo che l'acquisizione dei dati (Producer) e la loro elaborazione (Consumer) potessero scalare indipendentemente.

2. Resilienza e "Defensive Programming":

Il sistema dimostra una capacità di sopravvivenza ai guasti parziali (Fault Tolerance) superiore a una implementazione standard. L'integrazione di pattern avanzati quali:

- o **Circuit Breaker Custom:** Ha protetto le risorse interne in caso di latenza dell'API OpenSky, implementando una logica *Fail-Fast* con recupero automatico.
- o **Active Waiting & Graceful Degradation:** La gestione intelligente dello startup e l'uso di dati *Mock* in caso di fallimento esterno assicurano la continuità del sistema anche in condizioni degradate.
- o **Gestione della Concorrenza:** L'uso di cache thread-safe (threading.Lock) nello User Manager garantisce la consistenza dei dati anche sotto stress test concorrenti.

3. Analisi rispetto al Teorema CAP

Il sistema adotta un approccio ibrido rispetto al Teorema CAP, segmentando le garanzie in base al dominio del microservizio:

- **User Manager (CP - Consistency/Partition Tolerance):** Privilegia la consistenza (ACID) grazie a PostgreSQL. In caso di partizione di rete verso il DB, il servizio sacrifica la disponibilità (rifiuta registrazioni) pur di non creare utenti duplicati o corrotti.
- **Data Collector (AP - Availability/Partition Tolerance):** Privilegia la disponibilità. Grazie al pattern *Circuit Breaker* e all'uso di Dati Mock, il sistema continua a rispondere alle richieste di lettura anche se la fonte dati esterna (OpenSky) o il network sono compromessi, accettando temporaneamente una minore accuratezza del dato.

4. Integrità dei Dati e Semantica di Consegna:

La sfida della duplicazione dei messaggi è stata risolta implementando una semantica At-Least-Once Stretto lato Kafka (con flush manuale e commit sincrono) combinata con una logica di Idempotenza Applicativa (deduplica basata su hash MD5) nel Notifier. Questa architettura ibrida ha permesso di ottenere la garanzia che nessun allarme venga mai perso, accettando un overhead minimo per filtrare i duplicati, un trade-off ottimale per un sistema di alerting critico.

In conclusione, **il progetto** non è solo un aggregatore di tecnologie, ma un'architettura validata che dimostra come gestire correttamente lo stato, la rete e i fallimenti in un ambiente distribuito moderno.